

JPML_TF Rebuild Notes

Haowu Duan

Contents

Chapter 1

Introduction

This document is a working log of the from-scratch TensorFlow rebuild of the JPMLCOE codebase under `code_clean/jpml_tf/`. It serves three purposes:

1. Track what was built at each phase, in build order, with the exact files involved.
2. Explain every new TensorFlow or TensorFlow Probability command the first time it appears, in plain language. Standard library functions (`json`, `pathlib`, etc.) are not explained.
3. Capture the test command and test result for every stage, with a short justification of why the result demonstrates the code is correct at that point.

The plan being executed is in `rebuilt_plan/00_overview.md` through `08_migration.md`. Each chapter below corresponds to one phase in that plan; sections within a chapter correspond to incremental sub-pieces inside the phase.

Environment. Python 3.12, TensorFlow 2.16.2, TensorFlow Probability 0.24.0, Pydantic 2.13.3, pytest 9.0.2, PyYAML 6.0.3. Package manager: `uv` 0.11.8. Virtualenv at `code/.venv/`. The `code_clean/` tree is the new code root; the legacy `code/` tree is untouched.

Convention. Code listings show the file path immediately above the listing. New TensorFlow commands appear in blue boxes labelled *New command*. Test results appear in green boxes that include the exact `pytest` command, the captured output, and a short justification.

Chapter 2

Phase 01 — Foundation, Config, Results Sink

Phase 01 builds the package import boundary, the global TensorFlow startup hook that pins `float64`, the Pydantic-validated scenario schema, and the uniform JSON test-result sink. Every later phase loads through these.

This chapter is built in sub-pieces. Each sub-piece introduces the smallest possible change, has its own gate test, and is documented before the next one starts.

2.1 Sub-piece 1.1 — Package import boundary

Goal

Establish the package root `jpml_tf/` so that `import jpml_tf` succeeds and has no import side effects, in particular it does not import TensorFlow. The dtype-policy invariant that later phases depend on requires that no TF code runs before `configure_tf()` is called; that invariant is impossible to maintain unless the package root stays empty of computational imports.

Files built

Listing 2.1: `code_clean/jpml_tf/__init__.py`

```
"""jpml_tf -- TensorFlow rebuild of JPMLCOE.

The package root is intentionally empty: importing ‘jpml_tf’ must not
trigger TensorFlow imports, scenario loading, or array creation. Every
entry point (CLI, test, notebook) calls ‘jpml_tf.startup.configure_tf()’
explicitly before any computational module is imported.
"""

__version__ = "0.1.0"
```

Listing 2.2: `code_clean/jpml_tf/types.py`

```
"""Shared cross-module types.

Populated incrementally as later phases need them. Phase 01 only owns
the package scaffold and does not yet require any types.
```

```
"""
```

Listing 2.3: code_clean/tests/conftest.py

```
"""pytest configuration shared by all tests under ‘code_clean/tests/‘.

Adds the package parent (‘code_clean/‘) to ‘sys.path‘ so tests can
‘import jpml_tf‘ without installing the package.
"""

import sys
from pathlib import Path

_PKG_PARENT = Path(__file__).resolve().parent.parent # code_clean/
if str(_PKG_PARENT) not in sys.path:
    sys.path.insert(0, str(_PKG_PARENT))
```

Listing 2.4: code_clean/tests/test_01_imports.py (excerpt)

```
def test_jpml_tf_imports() -> None:
    import jpml_tf # noqa: F401

def test_jpml_tf_exposes_version_string() -> None:
    import jpml_tf

    assert isinstance(jpml_tf.__version__, str)
    assert len(jpml_tf.__version__) > 0

def test_jpml_tf_does_not_import_tensorflow() -> None:
    for mod in [m for m in sys.modules
                 if m == "tensorflow" or m.startswith("tensorflow.")]:
        del sys.modules[mod]

    import jpml_tf # noqa: F401

    tf_loaded = [m for m in sys.modules
                  if m == "tensorflow" or m.startswith("tensorflow.")]
    assert tf_loaded == [], (
        f"Importing jpml_tf must not load TensorFlow, but found: {
            tf_loaded[:5]}"
    )
```

New commands introduced

This sub-piece introduces no TensorFlow or TensorFlow Probability commands. The only library-level facts in play are standard Python: `sys.modules` (the dictionary of every module Python has imported in the current process) and `Path(__file__).resolve().parent` (the absolute directory containing the current source file).

Test command and result

```
cd code_clean
python -m pytest tests/test_01_imports.py -v
```

Test result: 3 passed, 0.01s

```
tests/test_01_imports.py::test_jpml_tf_imports
    PASSED [ 33%]
tests/test_01_imports.py::test_jpml_tf_exposes_version_string
    PASSED [ 66%]
tests/test_01_imports.py::test_jpml_tf_does_not_import_tensorflow
    PASSED [100%]
===== 3 passed in 0.01s
=====
```

Why this result is sufficient

The first two tests are smoke checks: the package imports and exposes its version string. They would fail if `__init__.py` had a syntax error or if `__version__` were misspelled.

The third test is the load-bearing one. It clears any TensorFlow modules from `sys.modules`, imports `jpml_tf`, and then asserts that no TensorFlow module re-appeared. This proves that the package root has no transitive import path to TensorFlow. As long as this test passes on every change, the dtype-policy invariant introduced in the next sub-piece (configure `float64` before TF creates any tensor) cannot be silently broken by an accidental import.

There are no JSON-saved numerical results yet because the uniform result sink is built later in this phase (sub-piece 1.3). Once the sink exists, all phase tests — including these three — will route their results through it.

2.2 Sub-piece 1.2 — TensorFlow startup hook

Goal

Provide a single function “`configure_tf()`” that callers (CLI, test, notebook) invoke once per process, before any computational module of `jpml_tf` is imported and before any TensorFlow tensor is created. The function locks the global numeric mode to `float64`, configures device visibility, and optionally enables deterministic ops. Phase 01 of the rebuild plan locks `float64` as the only critical-path dtype; this sub-piece is where that lock physically lives.

Files built

Listing 2.5: `code_clean/jpml_tf/startup.py`

```
"""Process-level TensorFlow configuration."""

from __future__ import annotations

import os

_CONFIGURED = False
```

```

def configure_tf(
    *,
    force_cpu: bool = False,
    allow_gpu_growth: bool = True,
    deterministic_ops: bool = False,
    legacy_keras: bool = True,
) -> None:
    """Lock global TF numeric and device policy to rebuild defaults."""
    global _CONFIGURED

    os.environ["TF_USE_LEGACY_KERAS"] = "1" if legacy_keras else "0"

    import tensorflow as tf

    if force_cpu:
        tf.config.set_visible_devices([], "GPU")
    elif allow_gpu_growth:
        for gpu in tf.config.list_physical_devices("GPU"):
            try:
                tf.config.experimental.set_memory_growth(gpu, True)
            except RuntimeError:
                pass

    if deterministic_ops:
        tf.config.experimental.enable_op_determinism()

    tf.keras.backend.set_floatx("float64")
    tf.keras.mixed_precision.set_global_policy("float64")

    _CONFIGURED = True

def is_configured() -> bool:
    return _CONFIGURED

```

Listing 2.6: code_clean/tests/test_01_startup.py

```

import os

def test_configure_tf_sets_float64() -> None:
    from jpml_tf.startup import configure_tf
    configure_tf()

    import tensorflow as tf
    assert tf.keras.backend.floatx() == "float64"
    assert tf.keras.mixed_precision.global_policy().compute_dtype == "
        float64"

def test_configure_tf_is_idempotent() -> None:
    from jpml_tf.startup import configure_tf, is_configured

```

```

configure_tf()
configure_tf()
assert is_configured()

import tensorflow as tf
assert tf.keras.backend.floatx() == "float64"

def test_configure_tf_sets_legacy_keras_env_var() -> None:
    from jpml_tf.startup import configure_tf
    configure_tf(legacy_keras=True)
    assert os.environ.get("TF_USE_LEGACY_KERAS") == "1"

```

New commands introduced

New command: `tf.keras.backend.set_floatx`, `tf.keras.backend.floatx`

TensorFlow’s Keras submodule has a process-wide default float dtype called *floatx*. It controls the dtype of any Keras layer, Keras optimizer state, or anything that asks Keras “what’s your default float?” `set_floatx("float64")` flips that default; `floatx()` reads the current value. Default at TF startup is "float32".

Caveat to remember. *floatx* only governs Keras-aware code. Plain `tf.constant(1.0)` still gives `float32` regardless of *floatx*, because `tf.constant` infers dtype from the literal’s Python type. The rebuild’s discipline is to always pass explicit `dtype=tf.float64` when constructing tensors; setting *floatx* catches the Keras-side defaults that explicit-dtype discipline cannot reach.

New command: `tf.keras.mixed_precision.set_global_policy`, `.global_policy`

The mixed-precision API in Keras lets you say “compute in this dtype, store variables in that dtype.” Setting the *global policy* to "float64" forces Keras to use float64 for both compute and variables. It is a stronger lock than *floatx*: any Keras layer that asks “what compute dtype should I use?” gets “float64” from the policy, even if the layer was constructed with no explicit dtype argument.

Why both *floatx* and the policy. They control overlapping but not identical surfaces. *floatx* is the older API, used by some optimizers and helpers. The policy API is the newer, broader mechanism. Setting both belt-and-braces is cheap and avoids “it works in some versions of Keras but not others” surprises.

New	command:	<code>tf.config.list_physical_devices,</code> <code>tf.config.set_visible_devices</code>
------------	-----------------	---

`list_physical_devices("GPU")` returns the list of GPUs TF can see on this machine, as a list of `PhysicalDevice` objects.

`set_visible_devices([], "GPU")` hides all GPUs from TF for the rest of the process: every subsequent op runs on CPU. This is the clean way to force CPU-only mode — much more reliable than setting `CUDA_VISIBLE_DEVICES` after TF has already initialised.

Order matters. Both calls must happen before TF places any op on a device. After the

first computation runs, TF locks the device visibility for the process; later calls error out.

New command: `tf.config.experimental.set_memory_growth`

By default, TF eagerly allocates almost all of a GPU's memory the moment it touches that GPU. `set_memory_growth(gpu, True)` asks TF to grow its memory pool incrementally instead, which lets multiple TF processes share a GPU and prevents spurious OOM errors from one greedy process. The trade-off is a small fragmentation cost. The `experimental` prefix means the API name might change in a future TF version, but it has been in this form since TF 2.0.

New command: `tf.config.experimental.enable_op_determinism`

Some TF ops (e.g. cuDNN convolutions, segment reductions, parallel reductions) are intentionally non-deterministic so they can use faster parallel implementations. `enable_op_determinism()` forces TF to use deterministic implementations everywhere. The result: the same input plus the same seed always produces the same output, at the cost of slower training. Useful for debugging and for paper-quality reproducibility runs; disabled by default in the rebuild because most training is faster without it.

New command: `TF_USE_LEGACY_KERAS` environment variable

Not a TF command, but the only knob that controls which Keras backend TensorFlow uses. TF 2.16+ ships Keras 3 by default; setting `TF_USE_LEGACY_KERAS=1` switches it to the older Keras 2.x backend (`tf-keras`, installed separately). The rebuild defaults to legacy Keras for compatibility with the existing TFP HMC kernels, which still target the Keras 2.x backend on the office RTX 3090 box. TF reads this variable on its first import; setting it later has no effect.

Test command and result

```
cd code_clean
python -m pytest tests/ -v
```

Test result: 6 passed, 2.56s

```
tests/test_01_imports.py::test_jpml_tf_imports
PASSED
tests/test_01_imports.py::test_jpml_tf_exposes_version_string
PASSED
tests/test_01_imports.py::test_jpml_tf_does_not_import_tensorflow
PASSED
tests/test_01_startup.py::test_configure_tf_sets_float64
PASSED
tests/test_01_startup.py::test_configure_tf_is_idempotent
PASSED
tests/test_01_startup.py::test_configure_tf_sets_legacy_keras_env_var
```

PASSED

```
===== 6 passed, 2 warnings in 2.56s
=====
```

Why this result is sufficient

The first new test asserts the two complementary dtype locks took effect: `floatx()` is "float64" *and* the mixed-precision compute dtype is "float64". Either one alone would leave gaps that the other closes; both together cover every Keras code path the rebuild touches.

The second test asserts idempotence. Real entry points may call `configure_tf()` more than once — a script that imports a test fixture which itself calls it, for example. Idempotence guarantees this is safe: nothing throws, and the dtype stays locked.

The third test asserts that `TF_USE_LEGACY_KERAS=1` actually landed in `os.environ`. This is a surface check: the env var is the only public signal that controls Keras-2 versus Keras-3 mode, and later phases assume Keras 2 semantics for layer dtype propagation.

The test result is currently captured only as pytest stdout in this document. Sub-piece 1.3 builds the uniform JSON result sink; once it exists, all six tests above will be re-run with their assertions recorded as JSON records keyed by test name.

State of code_clean/ after this sub-piece

```
code_clean/
  jpml_tf/
    __init__.py
    startup.py          (new)
    types.py
  tests/
    __init__.py
    conftest.py
    test_01_imports.py
    test_01_startup.py  (new)
  notes/
    notes.tex           (this document)
    notes.pdf           (compiled output)
```